Avaliação Comparativa de Produtos com AHP-TOPSIS

A integração AHP-TOPSIS segue uma lógica de **duas fases sequenciais**: o **AHP** determina os **pesos dos critérios** (com validação estatística de consistência), e o **TOPSIS** usa esses pesos para **ranquear as alternativas** (produtos) por proximidade geométrica a soluções ideais.

FASE 1 — AHP: Determinação dos Pesos dos Critérios

Passo 1.1 — Estruturação hierárquica
Define-se o objetivo (ex: "melhor produto"), os critérios (ex: preço, qualidade, durabilidade, suporte) e as alternativas (produtos a comparar).

Passo 1.2 — Matriz de comparação pareada
Constrói-se uma matriz quadrada A (n × n, n = nº de critérios), onde a_ij representa a importância relativa do critério i sobre o critério j, usando a **escala de Saaty (1–9)**:

Valor	Significado
1	Igual importância
3	Importância moderada
5	Importância forte
7	Importância muito forte
9	Importância extrema
2,4,6,8	Valores intermediários

Propriedade: **a_ji = 1/a_ij e a_ii = 1**.

Passo 1.3 — Cálculo do vetor de prioridades (pesos)
Método da média geométrica (mais simples e estável que o autovetor exato):

GM_i = (produto de a_ij para j=1..n)^(1/n)

$$w_i = GM_i \,/\, \text{soma}(GM_1 \ldots GM_n)$$

O vetor w = [w_1, w_2, ..., w_n] são os pesos normalizados dos critérios (soma = 1).

Passo 1.4 — Validação da consistência lógica

1. Calcular o vetor Aw (matriz original × vetor de pesos).
2. Calcular λmax:

$$\lambda max = (1/n) * \text{soma}(\ (Aw)_i \,/\, w_i\ )$$

3. Índice de Consistência:

$$CI = (\lambda max - n) \,/\, (n - 1)$$

4. Índice Aleatório (RI) — tabela de Saaty conforme n:

n	1	2	3	4	5	6	7	8	9	10
RI	0	0	0,58	0,90	1,12	1,24	1,32	1,41	1,45	1,49

5. Razão de Consistência:

$$RC = CI / RI$$

Critério de aceitação: se RC ≤ 0,10, os julgamentos são consistentes e os pesos são válidos. Caso contrário, a matriz de comparação deve ser revisada e refeita.

FASE 2 — TOPSIS: Ranqueamento das Alternativas

Passo 2.1 — Matriz de decisão
Constrói-se a matriz X (m alternativas × n critérios), com os valores de desempenho x_ij de cada produto i em cada critério j.

Passo 2.2 — Normalização vetorial

$$r_ij = x_ij / sqrt(soma(x_kj^2 \text{ para } k=1..m))$$

Isso elimina diferenças de unidade/escala entre critérios.

Passo 2.3 — Matriz normalizada ponderada
Aplicam-se os pesos w_j obtidos no AHP:

$$v_ij = w_j * r_ij$$

Passo 2.4 — Soluções ideal positiva e ideal negativa

Para cada critério, classificar como **benefício** (quanto maior, melhor — ex: qualidade) ou **custo** (quanto menor, melhor — ex: preço):

A+ = { max(v_ij) se benefício, min(v_ij) se custo } → solução ideal positiva

A- = { min(v_ij) se benefício, max(v_ij) se custo } → solução ideal negativa

Passo 2.5 — Distâncias euclidianas
Para cada alternativa i:

$$S+_i = sqrt(soma((v_ij - A+_j)^2 \text{ para } j=1..n))$$

$$S-_i = sqrt(soma((v_ij - A-_j)^2 \text{ para } j=1..n))$$

Passo 2.6 — Coeficiente de proximidade relativa

$$C^*_i = S-_i / (S+_i + S-_i)$$

C*_i varia entre 0 e 1: quanto mais próximo de 1, mais próxima a alternativa está da solução ideal.

Passo 2.7 — Ranqueamento
Ordenar as alternativas em ordem decrescente de C*_i. O produto com maior C* é a melhor opção segundo os critérios ponderados.

Algoritmo integrado (pseudocódigo resumido)

ENTRADA: matriz de comparação pareada (critérios), matriz de decisão (produtos x critérios)

```
1. w ← calcular_pesos_AHP(matriz_pareada)
2. RC ← calcular_consistencia(matriz_pareada, w)
3. SE RC > 0.10:
       RETORNAR erro "revisar julgamentos"
4. X_norm ← normalizar_vetorial(matriz_decisao)
5. V ← ponderar(X_norm, w)
6. A_mais, A_menos ← determinar_ideais(V, tipos_criterio)
7. PARA cada alternativa i:
       S_mais[i] ← distancia_euclidiana(V[i], A_mais)
       S_menos[i] ← distancia_euclidiana(V[i], A_menos)
       C[i] ← S_menos[i] / (S_mais[i] + S_menos[i])
8. RETORNAR ranking ordenado por C decrescente
```

Por que integrar os dois métodos

- **AHP** trata da **subjetividade estruturada**: converte julgamentos humanos qualitativos em pesos numéricos, com garantia estatística de coerência lógica (RC).
- **TOPSIS** trata da **objetividade geométrica**: usa dados de desempenho quantitativos para posicionar cada alternativa no espaço multicritério e medir sua distância às soluções extremas.
- A combinação evita as fraquezas isoladas de cada método: o **AHP** sozinho não lida bem com muitas alternativas (cresce exponencialmente o número de comparações), e o **TOPSIS** sozinho não tem mecanismo interno para definir pesos de forma consistente — ele normalmente os recebe prontos, e é aí que o AHP entra.

Simulação: Escolha de Notebook — 5 Produtos × 8 Critérios

Cenário

Critérios: Preço, Desempenho (CPU), RAM, Armazenamento, Bateria, Qualidade da Tela, Peso/Portabilidade, Suporte/Garantia
Alternativas: Produtos A, B, C, D, E

FASE 1 — AHP (pesos dos critérios)

Um decisor fictício preencheu a matriz de comparação pareada 8×8 (escala de Saaty). Aplicando a média geométrica e normalizando, os pesos resultantes foram:

Critério	Peso (w)
Desempenho (CPU)	0,2974
RAM	0,1960
Preço	0,1317
Armazenamento	0,1165
Tela	0,1068
Bateria	0,0683
Peso/Portabilidade	0,0424
Suporte/Garantia	0,0409

Validação de consistência:

- λmax = 8,1467
- CI = 0,0210
- RI (n=8) = 1,41
- **RC = 0,0149** → ✅ Consistente (RC ≤ 0,10). Julgamentos aceitos, pesos válidos para uso.

FASE 2 — TOPSIS (ranqueamento dos produtos)

Matriz de decisão (dados brutos)

Produto	Preço (R$)	Desempenho	RAM (GB)	Armaz. (GB)	Bateria (h)	Tela (1–10)	Peso (kg)	Suporte (anos)
A	4.200	8.500	16	512	9	8	1,4	2
B	3.600	6.200	8	256	11	6	1,8	1
C	5.300	9.800	32	1.024	7	9	1,6	3
D	3.900	7.000	16	512	10	7	1,3	2
E	4.800	9.100	16	512	8	9	1,9	3

*Preço e Peso são critérios de **custo** (menor = melhor); os demais são de **benefício** (maior = melhor).*

Soluções ideais (após normalização vetorial + ponderação)

- **A^+ (ideal positiva):** melhor valor de cada critério ponderado
- **A^- (ideal negativa):** pior valor de cada critério ponderado

Distâncias euclidianas e coeficiente de proximidade

Produto	S^+ (dist. ao ideal)	S^- (dist. ao anti-ideal)	$C^* = S^-/(S^+ + S^-)$
C	0,0266	0,1418	**0,8419**
E	0,0878	0,0680	0,4365
A	0,0884	0,0607	0,4068
D	0,0972	0,0506	0,3423
B	0,1419	0,0264	0,1570

Ranking Final

Posição	Produto	C*	Leitura
🥇 **1º**	**Produto C**	0,8419	Mais próximo do ideal — maior RAM, maior armazenamento, melhor desempenho, mais garantia; preço mais alto não penaliza, pois seu peso (0,13) é menor que o de desempenho (0,30)
🥈 **2º**	Produto E	0,4365	Bom desempenho e tela, mas peso alto e bateria fraca puxam para baixo
3º	Produto A	0,4068	Equilibrado, sem se destacar em nenhum critério de peso alto
4º	Produto D	0,3423	Mais barato e leve, mas fica atrás nos critérios de maior peso (desempenho, RAM)
5º	Produto B	0,1570	Mais barato, porém fraco nos dois critérios mais importantes (desempenho e RAM) — maior distância do ideal

Conclusão: o **Produto C** é a melhor alternativa segundo os critérios ponderados pelo decisor, pois combina os atributos de maior peso (Desempenho e RAM) com bom suporte, mesmo tendo o maior preço nominal — mostrando como a ponderação AHP evita que critérios de menor importância (como preço, aqui) dominem a decisão de forma desproporcional.

Implementação AHP-TOPSIS em Kotlin, espelhando exatamente os cálculos da simulação anterior (mesmos 8 critérios, 5 produtos e valores).

Estrutura do código:

- **AhpTopsis.calcularAHP()** — recebe a matriz de comparação pareada, calcula os pesos por média geométrica e valida a consistência (RC), lançando exceção se a matriz não for recíproca.
- **AhpTopsis.calcularTOPSIS()** — recebe a matriz de decisão, os pesos e os tipos de critério (BENEFICIO/CUSTO), retornando distâncias euclidianas e o ranking por C*.
- **main()** — reproduz o cenário do notebook (5 produtos × 8 critérios) e imprime pesos, verificação de consistência, tabela TOPSIS e ranking final.

```
// Ahp_Topsis.kt

import kotlin.math.pow
import kotlin.math.sqrt

/**
 * Implementação do método integrado AHP-TOPSIS para avaliação
 * comparativa multicritério de produtos.
 *
 * FASE 1 (AHP): calcula os pesos dos critérios a partir de uma matriz
 *               de comparação pareada e valida a consistência lógica (RC <= 0.10).
 * FASE 2 (TOPSIS): ranqueia as alternativas por distância euclidiana
 *                  às soluções ideais (positiva e negativa).
 */

enum class TipoCriterio { BENEFICIO, CUSTO }

/** Tabela de Índice Aleatório (RI) de Saaty, indexada pelo tamanho da matriz (n). */
private val TABELA_RI = mapOf(
    1 to 0.0, 2 to 0.0, 3 to 0.58, 4 to 0.90, 5 to 1.12,
    6 to 1.24, 7 to 1.32, 8 to 1.41, 9 to 1.45, 10 to 1.49
)

data class ResultadoAHP(
    val pesos: DoubleArray,
    val lambdaMax: Double,
    val ci: Double,
    val ri: Double,
    val rc: Double,
    val consistente: Boolean
)

data class ResultadoTOPSIS(
    val distanciaPositiva: DoubleArray,
    val distanciaNegativa: DoubleArray,
    val proximidadeRelativa: DoubleArray,
    val ranking: List<Pair<Int, Double>> // índice da alternativa -> C*
)
```

```
object AhpTopsis {

    /**
     * Calcula os pesos dos critérios via método da média geométrica
     * e valida a consistência lógica dos julgamentos (RC).
     *
     * @param matrizComparacao matriz quadrada n x n de comparação pareada (escala de
Saaty 1-9)
     * @throws IllegalArgumentException se a matriz não for recíproca ou não for
quadrada
     */
    fun calcularAHP(matrizComparacao: Array<DoubleArray>): ResultadoAHP {
        val n = matrizComparacao.size
        require(matrizComparacao.all { it.size == n }) { "A matriz de comparação deve
ser quadrada." }
        validarReciprocidade(matrizComparacao)

        // Passo 1: média geométrica de cada linha
        val mediasGeometricas = DoubleArray(n) { i ->
            var produto = 1.0
            for (j in 0 until n) produto *= matrizComparacao[i][j]
            produto.pow(1.0 / n)
        }

        // Passo 2: normalização -> vetor de pesos
        val somaMedias = mediasGeometricas.sum()
        val pesos = DoubleArray(n) { i -> mediasGeometricas[i] / somaMedias }

        // Passo 3: cálculo de λmax
        val aVezesW = DoubleArray(n) { i ->
            var soma = 0.0
            for (j in 0 until n) soma += matrizComparacao[i][j] * pesos[j]
            soma
        }
        val lambdaMax = aVezesW.indices.sumOf { i -> aVezesW[i] / pesos[i] } / n

        // Passo 4: Índice de Consistência (CI) e Razão de Consistência (RC)
        val ci = if (n > 1) (lambdaMax - n) / (n - 1) else 0.0
        val ri = TABELA_RI[n] ?: throw IllegalArgumentException(
            "Não há Índice Aleatório (RI) tabelado para n=$n (máximo suportado: 10)."
        )
        val rc = if (ri == 0.0) 0.0 else ci / ri

        return ResultadoAHP(
            pesos = pesos,
            lambdaMax = lambdaMax,
            ci = ci,
            ri = ri,
            rc = rc,
            consistente = rc <= 0.10
        )
    }
```

```
/*
* Verifica se a matriz de comparação é reciprocamente consistente: a_ji = 1 / a_ij.
*/
    private fun validarReciprocidade(matriz: Array<DoubleArray>, tolerancia: Double =
1e-6) {
        val n = matriz.size
        for (i in 0 until n) {
            for (j in 0 until n) {
                val esperado = 1.0 / matriz[j][i]
                require(kotlin.math.abs(matriz[i][j] - esperado) < tolerancia) {
                    "Matriz não recíproca na posição [$i][$j]: esperado
${"%.4f".format(esperado)}, " +
                            "encontrado ${matriz[i][j]}"
                }
            }
        }
    }

    /**
     * Ranqueia as alternativas por proximidade relativa às soluções ideais (TOPSIS).
     *
     * @param matrizDecisao matriz m x n (m alternativas, n critérios) com valores de
desempenho
     * @param pesos vetor de pesos dos critérios (deve somar 1, tipicamente vindo do
AHP)
     * @param tipos vetor indicando se cada critério é BENEFICIO (maior melhor) ou
CUSTO (menor melhor)
     */
    fun calcularTOPSIS(
        matrizDecisao: Array<DoubleArray>,
        pesos: DoubleArray,
        tipos: Array<TipoCriterio>
    ): ResultadoTOPSIS {
        val m = matrizDecisao.size          // nº de alternativas
        val n = matrizDecisao[0].size       // nº de critérios
        require(pesos.size == n) { "Nº de pesos deve ser igual ao nº de critérios." }
        require(tipos.size == n) { "Nº de tipos deve ser igual ao nº de critérios." }

        // Passo 1: normalização vetorial de cada coluna (critério)
        val normas = DoubleArray(n) { j ->
            sqrt((0 until m).sumOf { i -> matrizDecisao[i][j].pow(2) })
        }
        val matrizNormalizada = Array(m) { i ->
            DoubleArray(n) { j -> matrizDecisao[i][j] / normas[j] }
        }

        // Passo 2: matriz normalizada ponderada
        val v = Array(m) { i ->
            DoubleArray(n) { j -> matrizNormalizada[i][j] * pesos[j] }
        }

        // Passo 3: soluções ideal positiva (A+) e ideal negativa (A-)
```

```
        val idealPositiva = DoubleArray(n) { j ->
            val coluna = (0 until m).map { i -> v[i][j] }
            if (tipos[j] == TipoCriterio.BENEFICIO) coluna.max() else coluna.min()
        }
        val idealNegativa = DoubleArray(n) { j ->
            val coluna = (0 until m).map { i -> v[i][j] }
            if (tipos[j] == TipoCriterio.BENEFICIO) coluna.min() else coluna.max()
        }

        // Passo 4: distâncias euclidianas às soluções ideais
        val distanciaPositiva = DoubleArray(m) { i ->
            sqrt((0 until n).sumOf { j -> (v[i][j] - idealPositiva[j]).pow(2) })
        }
        val distanciaNegativa = DoubleArray(m) { i ->
            sqrt((0 until n).sumOf { j -> (v[i][j] - idealNegativa[j]).pow(2) })
        }

        // Passo 5: coeficiente de proximidade relativa C*
        val proximidadeRelativa = DoubleArray(m) { i ->
            val soma = distanciaPositiva[i] + distanciaNegativa[i]
            if (soma == 0.0) 0.0 else distanciaNegativa[i] / soma
        }

        // Passo 6: ranking decrescente por C*
        val ranking = proximidadeRelativa
            .withIndex()
            .sortedByDescending { it.value }
            .map { it.index to it.value }

        return ResultadoTOPSIS(distanciaPositiva, distanciaNegativa,
proximidadeRelativa, ranking)
    }
}

/**
 * Demonstração: escolha de notebook entre 5 produtos, avaliados por 8 critérios.
 */
fun main() {
    val criterios = listOf(
        "Preço", "Desempenho (CPU)", "RAM", "Armazenamento",
        "Bateria", "Tela", "Peso/Portabilidade", "Suporte/Garantia"
    )
    val produtos = listOf("Produto A", "Produto B", "Produto C", "Produto D",
"Produto E")

    // ------------------------------------------------------------
    // FASE 1: AHP - matriz de comparação pareada dos critérios
    // (mesma ordem de criterios acima)
    // ------------------------------------------------------------
    val matrizComparacao = arrayOf(
        doubleArrayOf(1.0, 1.0/3, 1.0/2, 1.0, 2.0, 2.0, 3.0, 4.0),
        doubleArrayOf(3.0, 1.0, 2.0, 3.0, 4.0, 3.0, 5.0, 5.0),
        doubleArrayOf(2.0, 1.0/2, 1.0, 2.0, 3.0, 2.0, 4.0, 4.0),
        doubleArrayOf(1.0, 1.0/3, 1.0/2, 1.0, 2.0, 1.0, 3.0, 3.0),
```

```
        doubleArrayOf(1.0/2, 1.0/4, 1.0/3, 1.0/2, 1.0, 1.0/2, 2.0, 2.0),
        doubleArrayOf(1.0/2, 1.0/3, 1.0/2, 1.0, 2.0, 1.0, 3.0, 3.0),
        doubleArrayOf(1.0/3, 1.0/5, 1.0/4, 1.0/3, 1.0/2, 1.0/3, 1.0, 1.0),
        doubleArrayOf(1.0/4, 1.0/5, 1.0/4, 1.0/3, 1.0/2, 1.0/3, 1.0, 1.0)
    )

    val resultadoAHP = AhpTopsis.calcularAHP(matrizComparacao)

    println("=== PESOS DOS CRITÉRIOS (AHP) ===")
    criterios.forEachIndexed { i, nome ->
        println("%-20s: %.4f".format(nome, resultadoAHP.pesos[i]))
    }
    println("Soma dos pesos: %.4f".format(resultadoAHP.pesos.sum()))
    println()
    println("=== VALIDAÇÃO DE CONSISTÊNCIA ===")
    println("λmax = %.4f".format(resultadoAHP.lambdaMax))
    println("CI   = %.4f".format(resultadoAHP.ci))
    println("RI   = %.2f".format(resultadoAHP.ri))
    println("RC   = %.4f -> %s".format(
        resultadoAHP.rc,
        if (resultadoAHP.consistente) "CONSISTENTE (RC <= 0.10)" else "INCONSISTENTE,
revisar matriz"
    ))
    println()

    if (!resultadoAHP.consistente) {
        println("Pesos rejeitados por inconsistência. Revise a matriz de comparação
pareada.")
        return
    }

    // ----------------------------------------------------------
    // FASE 2: TOPSIS - matriz de decisão dos produtos
    // Colunas: Preço | Desempenho | RAM | Armaz. | Bateria | Tela | Peso | Suporte
    // ----------------------------------------------------------
    val matrizDecisao = arrayOf(
        doubleArrayOf(4200.0, 8500.0, 16.0, 512.0,  9.0, 8.0, 1.4, 2.0), // Produto A
        doubleArrayOf(3600.0, 6200.0,  8.0, 256.0, 11.0, 6.0, 1.8, 1.0), // Produto B
        doubleArrayOf(5300.0, 9800.0, 32.0, 1024.0, 7.0, 9.0, 1.6, 3.0), // Produto C
        doubleArrayOf(3900.0, 7000.0, 16.0, 512.0, 10.0, 7.0, 1.3, 2.0), // Produto D
        doubleArrayOf(4800.0, 9100.0, 16.0, 512.0,  8.0, 9.0, 1.9, 3.0)  // Produto E
    )

    val tipos = arrayOf(
        TipoCriterio.CUSTO,     // Preço
        TipoCriterio.BENEFICIO, // Desempenho
        TipoCriterio.BENEFICIO, // RAM
        TipoCriterio.BENEFICIO, // Armazenamento
        TipoCriterio.BENEFICIO, // Bateria
        TipoCriterio.BENEFICIO, // Tela
        TipoCriterio.CUSTO,     // Peso
        TipoCriterio.BENEFICIO  // Suporte
    )
```

```
    val resultadoTOPSIS = AhpTopsis.calcularTOPSIS(matrizDecisao, resultadoAHP.pesos,
tipos)

    println("=== RESULTADOS TOPSIS ===")
    println("%-12s%10s%10s%10s".format("Produto", "S+", "S-", "C*"))
    produtos.forEachIndexed { i, nome ->
        println("%-12s%10.4f%10.4f%10.4f".format(
            nome,
            resultadoTOPSIS.distanciaPositiva[i],
            resultadoTOPSIS.distanciaNegativa[i],
            resultadoTOPSIS.proximidadeRelativa[i]
        ))
    }
    println()

    println("=== RANKING FINAL ===")
    resultadoTOPSIS.ranking.forEachIndexed { posicao, (indice, cStar) ->
        println("${posicao + 1}º lugar: ${produtos[indice]}  (C* =
%.4f)".format(cStar))
    }
}
```